EDA Checklist

HumAID · Kaggle “NLP with Disaster Tweets” · CrisisLexT26

Comprehensive Exploratory Data Analysis Checklist for the Disaster Response Classifier Project
Covers: Universal Checks · HumAID-Specific · Kaggle-Specific · CrisisLexT26-Specific · Cross-Dataset Comparison

Table of Contents

Table of Contents 2

How To Use This Checklist 3

Universal Checks (All Three Datasets) 4

HumAID-Specific Checks 5

Kaggle “NLP with Disaster Tweets”-Specific Checks 6

CrisisLexT26-Specific Checks 7

Cross-Dataset Comparison Checks 8

How To Use This Checklist

The three datasets play different roles in the project (HumAID trains the primary classifier, the Kaggle dataset trains a separate binary pre-filter, and CrisisLexT26 is a held-out external test set that is never trained on), so the checks below are split into what applies to all three, what's specific to each dataset given its role, and what should be checked once all three individual EDAs are complete. Work through Section A for every dataset first, then the matching dataset-specific section, then Section E last.

Universal Checks (All Three Datasets)

SECTION A

Run every one of these on HumAID, the Kaggle dataset, and CrisisLexT26 individually before moving on to the dataset-specific sections.

- ☐ Row count matches what you expect (HumAID: 53.5k / 7.79k / 15.2k; Kaggle: your own split total, ~7,613 labeled rows; CrisisLexT26: ~250k across 26 event files).
- ☐ Check for missing/null values in the text field.
- ☐ Check for exact duplicate rows.
- ☐ Check for near-duplicate / retweet-chain content (not just exact string matches).
- ☐ Check for rows that become empty strings after cleaning (pure URL, pure @mention, pure emoji tweets).
- ☐ Check text encoding — look for un-escaped HTML entities (&, <, >) sitting literally in the text rather than converted to &, <, >. This needs an explicit unescaping step in preprocessing, separate from URL/mention stripping.
- ☐ Check for stripped punctuation/apostrophes already baked into the source text (e.g. “youre” instead of “you're”) — if the original collection already mangled contractions, don't apply an inconsistent “fix” across datasets.
- ☐ Tweet length distribution (character count and token count) — min, max, mean, 50th / 95th / 99th percentile.
- ☐ Vocabulary size and out-of-vocabulary (OOV) rate against whatever embeddings/tokenizer you're using.
- ☐ Language check — confirm English-only as expected; flag and count any non-English rows.
- ☐ Hashtag frequency — top N overall, and whether a handful of hashtags dominate (a sign of one viral event skewing the data).
- ☐ @mention frequency and URL frequency per row.
- ☐ Emoji usage frequency and diversity.
- ☐ Check for tweet ID uniqueness (no duplicate IDs pointing to different text/labels).
- ☐ Manually read 15–20 random rows, not just the first N rows shown by .head().

HumAID-Specific Checks

SECTION B

HumAID is the primary training set for the 11-class model — these checks focus on label quality and split integrity, since this dataset's labels are what the core classifier learns directly.

- ☐ Class distribution (count and %) for all 11 class_label values, per split.
- ☐ Majority-to-minority imbalance ratio.
- ☐ Cross-tabulate disaster event vs. label (event isn't a column by default in the merged HF version — check whether event-level metadata is available in the raw per-event files instead, and pull it in if so, since it matters for leakage checks).
- ☐ Confirm class distribution shape is consistent across train / validation / test splits.
- ☐ Confirm no duplicate/near-duplicate tweets leak across train / validation / test specifically (not just within one split).
- ☐ Flag any class under a minimum count threshold (e.g. under 500 examples) for imbalance handling later.
- ☐ Check whether dont_know_cant_judge and not_humanitarian rows look meaningfully different from each other on manual read (these are the two most ambiguous/overlapping categories).

Kaggle “NLP with Disaster Tweets”-Specific Checks

SECTION C

This dataset trains the upstream binary pre-filter (disaster-related vs. not) — these checks focus on the quality of that on/off-topic signal and its two extra metadata columns.

- ☐ Binary target balance (real disaster = 1 vs. not = 0) — confirm the actual ratio in your own split.
- ☐ Check the keyword column: how many unique keywords, how many rows have it missing/null, and whether any keyword is 100% associated with only one class (a shortcut the model could latch onto instead of learning real content).
- ☐ Check the location column: this field is free-text and notoriously messy (real places, joke answers, emojis, blank) — quantify how much of it is actually usable before deciding whether to use it at all.
- ☐ Read 15–20 examples of target=0 rows that contain disaster-sounding words (fire, explosion, crash) used non-literally — this is the exact hard-negative pattern this dataset is relied on to teach the pre-filter.
- ☐ Confirm you're only using train.csv for labeled data — the competition's test.csv has no labels (it's for the original Kaggle leaderboard submission), so don't accidentally treat it as your held-out test set.
- ☐ Check tweet length distribution separately for target=0 vs. target=1 — if one class is systematically longer/shorter, that's a shortcut risk worth knowing about upfront.

CrisisLexT26-Specific Checks

SECTION D

This dataset is a held-out external test set only — it is never trained or tuned on. These checks focus on making sure it's clean and consistent enough to serve as a fair, single-use robustness check.

- ☐ Confirm the file/schema is consistent across all 26 per-event CSVs before concatenating (same column names, same label formatting) — these were released as 26 separate files, and it's common for one or two to have a formatting quirk.
- ☐ Confirm whether full tweet text is included directly, or whether the file only has tweet IDs requiring rehydration via Twitter's API — check this before assuming the text column is ready to use.
- ☐ Check all three of its label dimensions separately: informativeness (on/off-topic), information type, and information source — don't only look at one and assume the others are equally clean.
- ☐ Distribution of the 26 events — confirm no single event dominates the combined set the way one hashtag storm could dominate HumAID.
- ☐ Era-specific vocabulary check: compare hashtag/slang patterns here (2012–13) against HumAID (2016–19) qualitatively, since this is exactly the distribution shift the later robustness test is trying to measure.
- ☐ Confirm event_name (or equivalent) is retained as a column after concatenating the 26 files, so results can still be broken down per-event later even though it's treated as one combined external test set.

Cross-Dataset Comparison Checks

SECTION E

Do these once all three individual EDAs above are complete — they compare the datasets against each other rather than looking at any one in isolation.

- ☐ Vocabulary overlap between HumAID and CrisisLexT26 — how much of CrisisLexT26's vocabulary is actually covered by whatever embeddings/tokenizer was trained on HumAID.
- ☐ Tweet length distribution side-by-side across all three (a simple overlaid histogram) — flags whether the max_len setting (chosen from HumAID alone) will silently truncate a lot of CrisisLexT26 or Kaggle text.
- ☐ Label taxonomy feasibility check: attempt a rough mapping from CrisisLexT26's information-type categories onto HumAID's 11 classes, and note upfront which categories don't map cleanly (this needs to be documented for the robustness-test write-up regardless of how clean the mapping turns out).
- ☐ Confirm preprocessing pipeline output looks equivalent across all three after cleaning (spot-check 5 cleaned rows from each side-by-side) — this is what makes any later score comparison fair rather than an artifact of inconsistent cleaning.